AXI

Complete AXI Protocol Guide

Everything You Need to Know About AMBA AXI

---

1. Introduction & Architecture

1.1 What is AXI?

AMBA AXI (Advanced eXtensible Interface) is a high-performance, high-frequency protocol designed for:

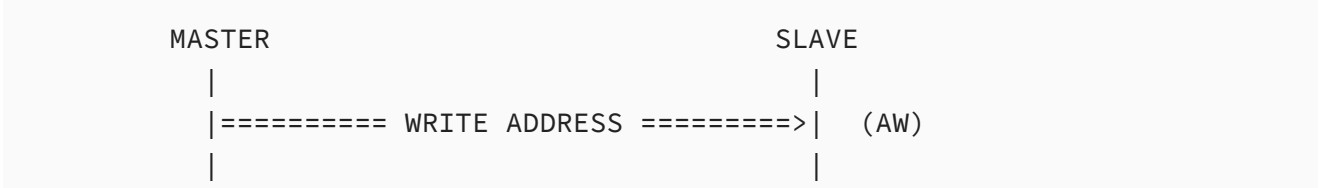
- **High bandwidth** and **low latency** designs
- **High-frequency operation** without complex bridges
- Meeting interface requirements of a **wide range of components**
- **Memory controllers** with high initial access latency (perfect for DDR!)

1.3 AXI Versions

- **AXI3** - Original specification (2003)
- **AXI4** - Enhanced version (2010) - **RECOMMENDED FOR NEW DESIGNS**
  - Burst length increased from 16 to 256 beats (individual data transfers in a burst)
  - Added Quality of Service (QoS) signaling (QoS = priority levels, High priority data goes first, low priority waits. Makes the **system** work better, not necessarily smoother for everyone).
  - Better cache support. region signals helps the **interconnect decide how to route or interpret the address**
  - Removed write data interleaving (**Transaction** = Complete operation (address → data → response). **AXI3:** Data could come **out of order**, mixed between different transactions, **AXI4:** Data must come **in order** for each transaction).
- **AXI4-Lite** - Simplified subset for simple peripherals (no bursts)

1.4 Core Architecture: 5 Independent Channels

AXI separates read and write transactions into **5 independent channels**:



```
            |========== WRITE DATA ============>|  (W)
            |                                    |
            |<========= WRITE RESPONSE =========|  (B)
            |                                    |
            |========== READ ADDRESS ==========>|  (AR)
            |                                    |
            |<========= READ DATA ==============|  (R)
            |                                    |
```

Why 5 channels?

- **Independence**: Read and write can operate simultaneously (full duplex)
- **Pipelining**: Address can be sent before previous data completes (ask Praveen sir how hazards r handled)
- **Performance**: Allows out-of-order completion (**IN (to slave):** Addresses = send freely, anytime, Write data = must be grouped per transaction (AXI4) **OUT (from slave):** Responses = can be out of order)
- **DRAM efficiency**: Perfect for hiding memory controller latency

1.5 Key Features

1. **Separate read and write channels** - Full duplex operation
2. **Burst-based transactions** - One address, multiple data beats
3. **Out-of-order transaction completion** - Using transaction IDs
4. **Support for unaligned transfers**
5. **Byte-level write strobes** - Partial word writes
6. **Multiple outstanding transactions** - Pipeline depth
7. **Easy addition of register stages** - For timing closure

---

2. Signal Descriptions

2.1 Global Signals

Signal	Source	Description
`ACLK`	Clock source	**Global clock signal** - All signals sampled on rising edge
`ARESETn`	Reset source	**Global reset signal** - Active LOW, asynchronous

Reset behavior:

- During reset (ARESETn = 0): All masters and slaves must drive VALID signals LOW
- No other reset requirements on signals

2.2 Write Address Channel (AW)

Purpose: Master sends write transaction address and control information to slave

Signal	Width	Source	Description
`AWID`	ID_WIDTH	Master	**Write address ID** - Identifies transaction
`AWADDR`	ADDR_WIDTH	Master	**Write address** - Address of first transfer in burst
`AWLEN`	8 bits (AXI4) / 4 bits (AXI3)	Master	**Burst length** - Number of transfers = AWLEN + 1
`AWSIZE`	3 bits	Master	**Burst size** - Bytes per beat (2^AWSIZE)
`AWBURST`	2 bits	Master	**Burst type** - FIXED(00), INCR(01), WRAP(10)
`AWLOCK`	2 bits (AXI3) / 1 bit (AXI4)	Master	**Lock type** - For atomic operations
`AWCACHE`	4 bits	Master	**Cache type** - Memory type and cacheability
`AWPROT`	3 bits	Master	**Protection type** - Privilege, security, access type
`AWQOS`	4 bits	Master	**Quality of Service** - Priority indicator (AXI4 only)
`AWREGION`	4 bits	Master	**Region identifier** - (AXI4 only)
`AWUSER`	USER_WIDTH	Master	**User-defined** - Optional custom signals (AXI4 only)
`AWVALID`	1 bit	Master	**Valid signal** - Address/control info is valid
`AWREADY`	1 bit	Slave	**Ready signal** - Slave can accept address

Key Details:

- **AWLEN encoding:**
  - AXI3: 0-15 (supports 1-16 beat bursts)
  - AXI4: 0-255 (supports 1-256 beat bursts)
- **AWSIZE encoding:** Bytes per transfer = 2^AWSIZE
  - 000 = 1 byte
  - 001 = 2 bytes
  - 010 = 4 bytes

- 011 = 8 bytes
- ... up to 111 = 128 bytes

2.3 Write Data Channel (W)

Purpose: Master sends write data to slave (can be before, during, or after address)

Signal	Width	Source	Description
`WID`	ID_WIDTH	Master	**Write data ID** - AXI3 ONLY (removed in AXI4)
`WDATA`	DATA_WIDTH	Master	**Write data** - Typically 32, 64, 128, 256, 512, 1024 bits
`WSTRB`	DATA_WIDTH/8	Master	**Write strobes** - Byte lane enables (1 bit per byte)
`WLAST`	1 bit	Master	**Write last** - Indicates last beat in burst
`WUSER`	USER_WIDTH	Master	**User-defined** - Optional (AXI4 only)
`WVALID`	1 bit	Master	**Valid signal** - Write data is valid
`WREADY`	1 bit	Slave	**Ready signal** - Slave can accept data

Critical Points:

- **WSTRB:** Each bit corresponds to one byte lane in WDATA
  - WSTRB[n] = 1: WDATA[(8n+7):(8n)] is valid
  - WSTRB[n] = 0: WDATA byte lane is ignored
- **WLAST:** MUST be asserted on the final data beat
- **AXI3 vs AXI4:** WID removed in AXI4 (write data ordering simplified)

2.4 Write Response Channel (B)

Purpose: Slave sends write transaction completion status to master

Signal	Width	Source	Description
`BID`	ID_WIDTH	Slave	**Response ID** - Must match AWID of the transaction
`BRESP`	2 bits	Slave	**Write response** - Transaction status
`BUSER`	USER_WIDTH	Slave	**User-defined** - Optional (AXI4 only)
`BVALID`	1 bit	Slave	**Valid signal** - Write response is valid

Signal	Width	Source	Description
`BREADY`	1 bit	Master	**Ready signal** - Master can accept response

BRESP Encoding:

- `00` = **OKAY** - Normal success
- `01` = **EXOKAY** - Exclusive access success
- `10` = **SLVERR** - Slave error (transaction failed)
- `11` = **DECERR** - Decode error (address invalid)

Important:

- Master MUST be able to accept write responses in **any order**
- Slave can send response before all data received (for buffered writes)

2.5 Read Address Channel (AR)

Purpose: Master sends read transaction address and control information to slave

Signal	Width	Source	Description
`ARID`	ID_WIDTH	Master	**Read address ID** - Identifies transaction
`ARADDR`	ADDR_WIDTH	Master	**Read address** - Address of first transfer in burst
`ARLEN`	8 bits (AXI4) / 4 bits (AXI3)	Master	**Burst length** - Number of transfers = ARLEN + 1
`ARSIZE`	3 bits	Master	**Burst size** - Bytes per beat (2^ARSIZE)
`ARBURST`	2 bits	Master	**Burst type** - FIXED(00), INCR(01), WRAP(10)
`ARLOCK`	2 bits (AXI3) / 1 bit (AXI4)	Master	**Lock type** - For atomic operations
`ARCACHE`	4 bits	Master	**Cache type** - Memory type and cacheability
`ARPROT`	3 bits	Master	**Protection type** - Privilege, security, access type
`ARQOS`	4 bits	Master	**Quality of Service** - Priority (AXI4 only)
`ARREGION`	4 bits	Master	**Region identifier** - (AXI4 only)
`ARUSER`	USER_WIDTH	Master	**User-defined** - Optional (AXI4 only)

Signal	Width	Source	Description
`ARVALID`	1 bit	Master	**Valid signal** - Address/control info is valid
`ARREADY`	1 bit	Slave	**Ready signal** - Slave can accept address

2.6 Read Data Channel (R)

Purpose: Slave sends read data and response to master

Signal	Width	Source	Description
`RID`	ID_WIDTH	Slave	**Read data ID** - Must match ARID of the transaction
`RDATA`	DATA_WIDTH	Slave	**Read data** - The actual data
`RRESP`	2 bits	Slave	**Read response** - Transaction status
`RLAST`	1 bit	Slave	**Read last** - Indicates last beat in burst
`RUSER`	USER_WIDTH	Slave	**User-defined** - Optional (AXI4 only)
`RVALID`	1 bit	Slave	**Valid signal** - Read data is valid
`RREADY`	1 bit	Master	**Ready signal** - Master can accept data

RRESP Encoding: (same as BRESP)

- `00` = **OKAY** - Normal success
- `01` = **EXOKAY** - Exclusive access success
- `10` = **SLVERR** - Slave error
- `11` = **DECERR** - Decode error

Critical:

- **RLAST:** MUST be asserted with the final data beat
- Master MUST be able to accept read data in **any order** (based on RID)

2.7 Signal Naming Convention

All AXI signals use a consistent naming pattern:

- **First letter:** A (address), W (write data), B (write response), R (read data)
- **Remainder:** Signal name (ADDR, DATA, VALID, READY, etc.)

Channel identification:

- AW* = Write address channel
- W* = Write data channel
- B* = Write response channel
- AR* = Read address channel
- R* = Read data channel

---

3. Channel Handshake Protocol

3.1 The VALID/READY Handshake

Every AXI channel uses a two-way handshake mechanism:

```
Source (Master/Slave):
- Asserts VALID when information is available
- Keeps VALID asserted until transfer completes
- Can assert VALID before READY

Destination (Slave/Master):
- Asserts READY when it can accept information
- Can assert READY before VALID
- Can wait for VALID before asserting READY

TRANSFER OCCURS:
- When BOTH VALID and READY are HIGH on rising clock edge
```

Timing Diagram:

```
Clock    __|‾|__|‾|__|‾|__|‾|__|‾|__|‾|__|‾|__

VALID    ________|‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾|______

READY    _________________|‾‾‾‾‾‾‾‾‾|_________

Transfer              NO   NO   YES  NO
```

3.2 Handshake Rules

CRITICAL RULES:

1. **VALID must NOT depend on READY**
   - Source cannot wait for READY before asserting VALID
   - Prevents circular dependencies and deadlock

2. **READY can depend on VALID**
   - Destination can wait to see VALID before asserting READY
   - Allows simple slave implementations
3. **Once VALID is asserted:**
   - Must remain HIGH until handshake completes
   - Associated signals (data, address, etc.) must remain stable
   - Cannot be retracted
4. **READY can be asserted/deasserted freely**
   - Can be HIGH before VALID
   - Can toggle while waiting for VALID (though unusual)

3.3 Three Handshake Scenarios

Scenario 1: VALID before READY

```
Clock    __|‾|__|‾|__|‾|__|‾|__

VALID    ____|‾‾‾‾‾‾‾‾‾|______

READY    ________|‾‾‾|_________

         Source ready first,
         waits for destination
```

Scenario 2: READY before VALID

```
Clock    __|‾|__|‾|__|‾|__|‾|__

VALID    ________|‾‾‾|_________

READY    ____|‾‾‾‾‾‾‾|_________

         Destination ready,
         waiting for source
```

Scenario 3: VALID and READY together

```
Clock    __|‾|__|‾|__|‾|__

VALID    ____|‾‾‾|_________

READY    ____|‾‾‾|_________

         Both ready same cycle
         (low latency)
```

3.4 Channel Relationships

Key principle: The 5 channels are **independent** but have ordering dependencies:

Write Transaction Dependencies:

```
AWVALID/AWREADY ──┐
                  ├──> Can happen in any order
WVALID/WREADY  ───┘

BVALID/BREADY  ─────> Must come AFTER write address accepted
```

Read Transaction Dependencies:

```
ARVALID/ARREADY ──> Must happen first

RVALID/RREADY  ─────> Must come AFTER read address accepted
```

3.5 Channel Handshake Dependencies (CRITICAL!)

Master requirements:

- Must NOT wait for AWREADY before asserting AWVALID
- Must NOT wait for WREADY before asserting WVALID
- Must NOT wait for ARREADY before asserting ARVALID
- CAN wait for BVALID before asserting BREADY (optional)
- CAN wait for RVALID before asserting RREADY (optional)

Slave requirements:

- CAN wait for AWVALID before asserting AWREADY (recommended)
- CAN wait for WVALID before asserting WREADY (recommended)
- CAN wait for ARVALID before asserting ARREADY (recommended)
- Must NOT wait for BREADY before asserting BVALID
- Must NOT wait for RREADY before asserting RVALID

Why these rules?

- Prevents deadlock
- Allows simple implementations
- Enables pipelining

---

4. Addressing and Burst Transactions

4.1 Burst Overview

A **burst** is a single transaction consisting of:

- **One address phase** (AWADDR/ARADDR)
- **Multiple data phases** (controlled by AWLEN/ARLEN)

Benefits for memory controllers:

- Amortizes address overhead
- Maps directly to DRAM prefetch architecture
- Enables efficient row buffer utilization

4.2 Burst Length (AWLEN/ARLEN)

Encoding: Number of data transfers = AWLEN/ARLEN + 1

Version	AWLEN/ARLEN Width	Burst Length Range
AXI3	4 bits	1-16 beats
AXI4	8 bits	1-256 beats

Examples:

- AWLEN = 0 → 1 data transfer
- AWLEN = 7 → 8 data transfers
- AWLEN = 15 → 16 data transfers (AXI3 max)
- AWLEN = 255 → 256 data transfers (AXI4 max)

AXI4 Restrictions:

- FIXED bursts: Max 16 beats
- WRAP bursts: 2, 4, 8, or 16 beats only

4.3 Burst Size (AWSIZE/ARSIZE)

Encoding: Bytes per transfer = 2^AWSIZE / 2^ARSIZE

AWSIZE/ARSIZE	Bytes per Transfer	Typical Use
3'b000	1 byte	8-bit transfers
3'b001	2 bytes	16-bit transfers
3'b010	4 bytes	32-bit transfers
3'b011	8 bytes	64-bit transfers
3'b100	16 bytes	128-bit transfers

AWSIZE/ARSIZE	Bytes per Transfer	Typical Use
3'b101	32 bytes	256-bit transfers
3'b110	64 bytes	512-bit transfers
3'b111	128 bytes	1024-bit transfers

Important:

- Maximum size = data bus width (in bytes)
- Narrow transfers possible (size < bus width)
- Size determines address alignment requirements

4.4 Burst Types (AWBURST/ARBURST)

Three burst types available:

4.4.1 FIXED Burst (2'b00)

Address remains constant for all beats

```
Address:  [ADDR] [ADDR] [ADDR] [ADDR]
Beat:        0      1      2      3
```

Use cases:

- FIFO access
- Repeated access to same location
- Memory-mapped peripheral registers

Example: AWADDR=0x1000, AWLEN=3, AWSIZE=2 (4 bytes), AWBURST=FIXED

- Beat 0: Address 0x1000
- Beat 1: Address 0x1000
- Beat 2: Address 0x1000
- Beat 3: Address 0x1000

4.4.2 INCR Burst (2'b01)

Address increments by size for each beat

```
Address:  [ADDR] [ADDR+SIZE] [ADDR+2*SIZE] [ADDR+3*SIZE]
Beat:        0        1            2             3
```

Use cases:

- Normal memory access

- DMA transfers
- Most common for DDR memory controllers

Example: ARADDR=0x1000, ARLEN=3, ARSIZE=3 (8 bytes), ARBURST=INCR

- Beat 0: Address 0x1000
- Beat 1: Address 0x1008
- Beat 2: Address 0x1010
- Beat 3: Address 0x1018

4.4.3 WRAP Burst (2'b10)

Address wraps at specific boundary (aligned to burst size)

```
Wrapping boundary = (burst_length × bytes_per_beat)

Address wraps when it would cross this boundary
```

Use cases:

- **Cache line fills** (most common)
- Circular buffers
- Optimized for critical word first

Restrictions:

- Burst length MUST be 2, 4, 8, or 16 beats
- Start address must be aligned to (burst_length × size)

Example: ARADDR=0x1018, ARLEN=3 (4 beats), ARSIZE=3 (8 bytes), ARBURST=WRAP

- Wrap boundary = 4 × 8 = 32 bytes = 0x20
- Aligned boundary starts at 0x1000 (0x1018 & ~0x1F)
- Beat 0: Address 0x1018
- Beat 1: Address 0x1000 (wraps!)
- Beat 2: Address 0x1008
- Beat 3: Address 0x1010

Why WRAP for caches?

- CPU misses cache line at address 0x1018 (needs this word NOW)
- Cache requests wrap burst starting at 0x1018
- Gets critical word first (0x1018), then wraps to fill rest (0x1000, 0x1008, 0x1010)
- CPU can continue while cache line completes

4.5 Burst Address Calculation

General formula for beat N:

```
Address_N = Start_Address + (N × 2^AWSIZE)  [for INCR]
Address_N = Start_Address                   [for FIXED]
Address_N = Wrap_Boundary + ((Start_Offset + N × 2^AWSIZE) MOD
Wrap_Length) [for WRAP]
```

Read Burst: Master sends address and control signals. Slave accepts address. Slave keeps RVALID LOW until data is ready. Then slave sends multiple data beats back. On the final data transfer, slave asserts RLAST HIGH to show last item is being transferred.

Overlapping Bursts: Master can send second burst address after slave accepts first address, before first burst finishes. This lets slave process second burst in parallel with completing first burst. Enables pipelining and hides latency.

Write Burst: Master sends address and control on write address channel. Then master sends data items on write data channel. When master sends last data item, WLAST goes HIGH. After slave accepts all data, slave sends write response back to master showing write transaction is complete.

---

5. Response Signaling

5.1 Response Types

All transactions generate a response from the slave:

BRESP/RRESP	Encoding	Name	Meaning
2'b00	OKAY	Normal access success	Transaction completed successfully
2'b01	EXOKAY	Exclusive access success	Exclusive access completed successfully
2'b10	SLVERR	Slave error	Slave encountered error (e.g., FIFO overflow)
2'b11	DECERR	Decode error	No slave at address, or access not allowed

5.2 Response Handling

Write responses:

- One BRESP per write transaction (not per beat)

- Slave can respond before all data received (buffered writes)
- Master must accept responses in any order (use BID)

Read responses:

- RRESP per read data beat
- All beats in burst typically have same RRESP
- If error occurs mid-burst, remaining beats can have SLVERR

Error handling:

- Master decides how to handle SLVERR/DECERR
- Typical actions: retry, report to software, abort operation

5.3 Exclusive Access Response (EXOKAY)

Used for atomic read-modify-write operations:

- OKAY: Exclusive access succeeded
- EXOKAY: Exclusive monitor state changed
- Only relevant for exclusive transactions (ARLOCK/AWLOCK)

---

6. Ordering Model & Transaction IDs

6.1 Why Transaction IDs?

Problem: Memory controller has high latency

- First read takes 30-40ns (DDR activate + CAS latency)
- Want to issue multiple reads while waiting

Solution: Transaction IDs

- Master tags each transaction with unique ID
- Slave can complete transactions **out of order**
- Responses carry matching ID so master knows which transaction completed

6.2 ID Field Widths

Signal	ID Field	Width
Write address	AWID	Configurable (typically 4-8 bits)
Write response	BID	Must match AWID width
Read address	ARID	Configurable (typically 4-8 bits)

Signal	ID Field	Width
Read data	RID	Must match ARID width

Note: AXI4 removed WID (write data ID) - write data now ordered by AWID only

6.3 ID Usage Example

```
Master issues:
  Read A (ARID=0, ARADDR=0x1000)
  Read B (ARID=1, ARADDR=0x2000)
  Read C (ARID=2, ARADDR=0x3000)

Memory controller (slave) can respond:
  Read C data (RID=2) ← fastest (row buffer hit)
  Read A data (RID=0) ← slower (different bank)
  Read B data (RID=1) ← slowest (bank conflict)

Master uses RID to route data correctly
```

6.4 Ordering Rules

6.4.1 Read Ordering

Transactions with SAME ID:

- Must be returned **in order** (ordered completion)
- Example: ARID=5, read A then read B → data A must return before data B

Transactions with DIFFERENT IDs:

- Can be returned **in any order** (out-of-order completion)
- Example: ARID=5 read A, ARID=7 read B → B data can return before A data

6.4.2 Write Ordering (AXI4)

Transactions with SAME ID:

- Write address order determines write data order
- Responses can be out of order
- Example: AWID=3, write A then write B → data A must go before data B

Transactions with DIFFERENT IDs:

- No ordering requirement between different IDs
- Complete flexibility for memory controller

AXI3 difference:

- WID allowed write data interleaving
- AXI4 removed this for simplicity

6.4.3 Read vs Write Ordering

No ordering relationship between reads and writes to different IDs

Exception: Software coherency concerns

- If read and write to same address, ID matching can enforce ordering
- Otherwise, software must use barriers/synchronization

6.5 Interconnect ID Handling

When multiple masters connect through interconnect:

- Interconnect may **append bits** to IDs to distinguish masters
- Example: 4-bit master IDs + 2-bit master number = 6-bit slave IDs
- Transparent to masters and slaves

6.6 Recommended ID Widths

Depends on pipelining depth:

- **4 bits** (16 IDs): Light pipelining
- **6 bits** (64 IDs): Moderate pipelining
- **8 bits** (256 IDs): Heavy pipelining

For memory controllers: 4-6 bits typical

---

7. Data Buses & Write Strobes

7.1 Data Bus Width

Common widths: 32, 64, 128, 256, 512, 1024 bits

AXI supports:

- 8 to 1024 bits (must be power of 2 in bytes: 8, 16, 32, 64, ... bits)
- Wider buses = higher bandwidth
- DDR controllers often use 128, 256, or 512-bit buses

7.2 Write Strobes (WSTRB)

Purpose: Byte-level write enables

Width: WSTRB has 1 bit per byte of WDATA

- 32-bit bus → 4-bit WSTRB
- 64-bit bus → 8-bit WSTRB
- 128-bit bus → 16-bit WSTRB

Encoding:

```
WSTRB[n] = 1 → Byte lane n is VALID (write this byte)
WSTRB[n] = 0 → Byte lane n is INVALID (don't write this byte)
```

Byte lane mapping:

```
WSTRB[0] controls WDATA[7:0]
WSTRB[1] controls WDATA[15:8]
WSTRB[2] controls WDATA[23:16]
...
WSTRB[n] controls WDATA[(8n+7):(8n)]
```

Example (32-bit bus):

```
WDATA = 0x12345678
WSTRB = 4'b1010

Written bytes: 0x12 at byte 3, 0x56 at byte 1
Ignored bytes: byte 2 and byte 0
```

Uses:

- **Partial word writes:** Update 1 byte in 4-byte word
- **Unaligned transfers:** First/last beat may not use full width
- **Byte enables from processor:** CPU byte enables pass through

Rules:

- WSTRB can be all zeros (no bytes written) - unusual but legal
- For read transactions, no byte strobes (always read full size)

7.3 Narrow Transfers

Definition: Transfer size is **less than** data bus width

Example: 64-bit data bus (8 bytes), AWSIZE=2 (4 bytes)

- Each beat only uses 4 bytes of the 8-byte bus
- Position depends on address

Address determines byte lanes used:

For 64-bit bus (8 bytes), AWSIZE=2 (4 bytes):

```
AWADDR[2:0] = 0 → Use byte lanes [3:0]
AWADDR[2:0] = 4 → Use byte lanes [7:4]
```

Narrow transfer formula:

Lower byte lane (start) = AWADDR[ADDR_WIDTH-1:0] MOD DATA_WIDTH_BYTES

Upper byte lane (end) = Lower + (2^AWSIZE - 1)

Why allow narrow transfers?

- Master may have narrow data path
- Simplifies master design
- Slave handles alignment

7.4 Byte Invariance

Critical principle: A byte's position depends ONLY on its address, not bus width

Example: Writing 0xAB to address 0x1003

- On 32-bit bus: Goes to byte lane determined by 0x1003[1:0] = 3
- On 64-bit bus: Goes to byte lane determined by 0x1003[2:0] = 3
- On 128-bit bus: Goes to byte lane determined by 0x1003[3:0] = 3

Implication: Same master/slave code works with different bus widths

Endianness:

- AXI is **little-endian** for byte addresses
- Byte 0 is LSB (least significant byte)
- Higher addresses → higher byte lanes

---

8. AXI4 Enhancements

8.1 Extended Burst Length

AXI3: 1-16 beats (AWLEN/ARLEN = 4 bits)
AXI4: 1-256 beats (AWLEN/ARLEN = 8 bits)

Why 256 beats?

- Modern memory controllers handle long bursts efficiently
- Reduces address overhead
- Example: 256 beats × 64 bytes = 16KB in one burst
- Perfect for large DMA transfers

Restrictions:

- FIXED bursts still limited to 16 beats
- WRAP bursts still 2, 4, 8, or 16 beats only

8.2 Quality of Service (QoS)

New signals: AWQOS[3:0], ARQOS[3:0]

Purpose: Priority/urgency indicator

- Higher value = higher priority
- 0 = no QoS, lowest priority
- 15 = highest priority

Use in memory controller:

- Critical transactions (real-time video) get high QoS
- Background DMA gets low QoS
- Memory controller scheduler uses QoS for arbitration

Example:

```
Read A: ARQOS=12 (high priority - display)
Read B: ARQOS=4  (normal - CPU)
Read C: ARQOS=0  (low - background DMA)

Memory controller serves: A → B → C
```

8.3 Region Identifiers

New signals: AWREGION[3:0], ARREGION[3:0]

Purpose: Multi-region addressing

- Allows single physical slave to decode up to 16 regions
- Each region can have different properties
- Optional feature (default=0)

Use case:

```
Memory controller with multiple memory types:
  Region 0: DDR (cacheable)
  Region 1: Flash (device memory)
  Region 2: SRAM (non-cacheable)
```

8.4 Removed Write Data Interleaving

AXI3: Allowed different write IDs on successive WDATA beats

- Complex: Required WID on every beat
- Hard to implement efficiently

AXI4: Removed WID, simplified write data ordering

- All WDATA for transaction must be contiguous
- Order determined by AWID transaction order
- Simpler slave implementation

Impact on memory controllers: Much simpler write data buffering

8.5 Write Response Dependencies (AXI4)

AXI3: Slave could send BVALID before all WDATA received (if buffered)

AXI4: Added dependency rule:

- Slave must not assert BVALID until:
  1. Last WDATA beat received (WLAST seen), AND
  2. Write successfully completed

Why?

- Prevents accepting write responses before write completes
- Clearer transaction boundaries
- Better error handling

Exception: Buffered writes

- Slave with deep write buffer can still respond early
- But must guarantee write will complete

8.6 Updated AWCACHE/ARCACHE

AXI4 adds more cache attributes:

Bit assignments:

- `[0]` - Bufferable (can buffer transaction)

- `[1]` - Modifiable (interconnect can modify transaction)
- `[2]` - Read Allocate (allocate on read miss)
- `[3]` - Write Allocate (allocate on write miss)

Memory types:

- `0000`, `0001`, `0010`, `0011` = Device (non-cacheable)
- `0010` = Normal Non-cacheable
- `0110` = Write-through cacheable
- `1110` = Write-back cacheable

For DDR: Typically use `1111` (write-back, read/write allocate)

8.7 User-Defined Signals

New optional signals: AWUSER, WUSER, BUSER, ARUSER, RUSER

Purpose: Custom sideband information

- Configurable width
- Can carry any application-specific data
- Examples:
  - Encryption keys
  - Security attributes
  - Debug info
  - Timestamp

Memory controller use:

- Pass through encryption enable
- ECC configuration
- Scrubbing hints

8.8 Simplified Atomic Accesses

AXI3: AWLOCK/ARLOCK = 2 bits (Normal, Exclusive, Locked)
AXI4: AWLOCK/ARLOCK = 1 bit (Normal, Exclusive only)

Locked accesses deprecated (complex, rarely used)

---

9. Memory Controller Specific Considerations

9.1 Why AXI is Perfect for DDR Controllers

DDR characteristics:

1. **High initial latency** (row activate + CAS ~30-40ns)
2. **Fast burst access** once row open
3. **Multiple banks** can operate in parallel
4. **Benefits from reordering** (row buffer locality)

AXI solutions:

1. **Out-of-order completion** hides latency
2. **Burst transactions** match DDR prefetch
3. **Multiple transaction IDs** enable bank parallelism
4. **Independent channels** allow read/write overlap

9.2 Mapping AXI to DDR Commands

Typical flow:

```
AXI Read Request:
  ARADDR=0x1000, ARLEN=7, ARSIZE=3, ARBURST=INCR

Memory Controller translates to DDR:
  1. Decode address → Bank, Row, Column
  2. Issue ACTIVATE (open row)
  3. Wait tRCD (RAS-to-CAS delay)
  4. Issue READ burst
  5. Wait CAS latency
  6. Capture data from PHY
  7. Stream RDATA back via AXI (8 beats)
```

9.3 Burst Length Matching

DDR burst length: Usually 8 (BL8) or 4 (BL4)

- DDR3/DDR4 typically BL8
- Each READ/WRITE command transfers 8 beats from DRAM core

AXI burst length: 1-256 beats

Controller must:

- Break long AXI bursts into multiple DDR bursts
- Combine short AXI bursts when beneficial
- Align to DDR burst boundaries for efficiency

Example:

```
AXI: ARLEN=15 (16 beats), ARSIZE=3 (8 bytes each) = 128 bytes
```

```
DDR: Issue 2× BL8 READ commands
  READ1: 8 beats × 8 bytes = 64 bytes
  READ2: 8 beats × 8 bytes = 64 bytes
  Total: 128 bytes
```

9.4 Address Translation

AXI address: Byte address (e.g., 0x12345678)

DDR address components:

- **Bank:** Which DDR bank (2-3 bits)
- **Row:** Which row in bank (13-16 bits)
- **Column:** Which column in row (10-11 bits)

Example mapping (DDR4):

```
AXI Address: [31:0]

Possible mapping:
  Column[9:3] = ADDR[9:3]    (8-byte aligned)
  Bank[1:0]   = ADDR[11:10]  (interleave banks)
  Row[15:0]   = ADDR[27:12]  (high order = row)

Different mappings optimize for:
  - Sequential access (group rows)
  - Parallel access (spread across banks)
  - Page hit rate (column locality)
```

9.5 Transaction Reordering Strategy

Goal: Maximize DDR efficiency

Reordering opportunities:

1. **Row buffer hits first:** Serve requests to open row
2. **Bank parallelism:** Issue to different banks simultaneously
3. **Read/write grouping:** Minimize mode switches (tWTR, tRTW)
4. **Age priority:** Don't starve old requests

Example reorder:

```
Incoming AXI requests:
  1. Read  Bank0 Row5  (ARID=0)
  2. Read  Bank1 Row7  (ARID=1)
  3. Read  Bank0 Row5  (ARID=2)  ← Same row!
  4. Write Bank2 Row3  (AWID=0)
```

```
Optimized order:
  - Activate Bank0 Row5
  - Issue Read #1, then Read #3 (row buffer hits)
  - Activate Bank1 Row7
  - Issue Read #2 (parallel bank)
  - Activate Bank2 Row3
  - Issue Write #4

Return order (using IDs):
  - RID=0, RID=2, RID=1, BID=0
```

9.6 Refresh Handling

DDR requires periodic refresh:

- Every row must refresh within tREFI (7.8µs typical)
- Refresh blocks normal access

AXI implications:

- During refresh, controller cannot accept new transactions
- Use ARREADY=0, AWREADY=0 to backpressure
- Outstanding reads/writes complete after refresh

Strategy:

- Track refresh timing
- Insert refresh between AXI transactions
- Minimize refresh blocking with smart scheduling

9.7 Write Buffering

Problem: AXI writes are bursty, DDR prefers aligned bursts

Solution: Write buffer in memory controller

- Collect AXI write data
- Combine into efficient DDR bursts
- Send BVALID early (buffered write response)

Example:

```
AXI Write: AWADDR=0x1004, AWLEN=0, AWSIZE=2 (1 word)
  → Buffer in controller

AXI Write: AWADDR=0x1008, AWLEN=0, AWSIZE=2 (1 word)
  → Combine in buffer
```

```
When buffer full or timeout:
  → Issue DDR WRITE with multiple words
  → More efficient than individual writes
```

9.8 Read Prefetching

Opportunity: DDR returns 8 beats per burst

If AXI requests 4 beats:

- Option 1: Only fetch 4 (underutilize DDR)
- Option 2: Fetch all 8, cache extra 4 (prefetch)

Prefetch benefits:

- Next sequential read may hit cache
- Better DDR utilization
- Hides latency

Considerations:

- Need prefetch buffer
- Track validity
- Handle coherency

9.9 Quality of Service Implementation

Use ARQOS/AWQOS for scheduling:

Example policy:

```
QoS Level  | Max Latency | Arbitration Weight
-----------|-------------|-------------------
15-12      | 100 cycles  | 8× priority
11-8       | 500 cycles  | 4× priority
7-4        | 2000 cycles | 2× priority
3-0        | Best effort | 1× priority
```

Implementation:

- Age counter per transaction
- QoS boosts urgency
- Prevent starvation with age limit

9.10 Error Handling

DDR errors to report via AXI:

1. **ECC uncorrectable error:**
   - Return RRESP=SLVERR on affected read
   - Master must decide: retry, report to SW, etc.
2. **Write to non-existent address:**
   - Return BRESP=DECERR
   - Can happen if address decode wrong
3. **Timeout:**
   - If DDR stuck, timeout transaction
   - Return SLVERR

9.11 4KB Boundary and DDR Pages

AXI 4KB rule aligns with DDR pages:

- DDR page size often 1-2KB per bank
- 4KB ensures no single burst crosses multiple banks/rows
- Simplifies controller address decode

Controller benefit:

- Single AXI burst stays in one row
- No mid-burst row changes
- Easier state machine

9.12 Power Management

AXI supports low-power:

- Clock gating when idle
- Deep power-down between transactions
- Wake-up latency hidden by ARREADY/AWREADY

Memory controller implementation:

- Track idle time
- Enter DDR self-refresh when no AXI activity
- Exit on new ARVALID/AWVALID

---

10. Quick Reference Tables

10.1 Signal Summary

Channel	Signals	Direction (Master→Slave)
Write Address	AWID, AWADDR, AWLEN, AWSIZE, AWBURST, AWLOCK, AWCACHE, AWPROT, AWQOS, AWREGION, AWUSER, AWVALID	→
Write Address	AWREADY	←
Write Data	WDATA, WSTRB, WLAST, WUSER, WVALID	→
Write Data	WREADY	←
Write Response	BVALID	←
Write Response	BID, BRESP, BUSER, BREADY	← →
Read Address	ARID, ARADDR, ARLEN, ARSIZE, ARBURST, ARLOCK, ARCACHE, ARPROT, ARQOS, ARREGION, ARUSER, ARVALID	→
Read Address	ARREADY	←
Read Data	RVALID	←
Read Data	RID, RDATA, RRESP, RLAST, RUSER, RREADY	← →
Global	ACLK, ARESETn	Clock/Reset source

10.2 Burst Type Encoding

AXBURST[1:0]	Type	Address Behavior
2'b00	FIXED	Constant address
2'b01	INCR	Incrementing address
2'b10	WRAP	Wrapping address
2'b11	Reserved	-

10.3 Response Encoding

XRESP[1:0]	Type	Description
2'b00	OKAY	Normal success
2'b01	EXOKAY	Exclusive success
2'b10	SLVERR	Slave error
2'b11	DECERR	Decode error

10.4 Size Encoding

AXSIZE[2:0]	Bytes	Bits	Typical Use
3'b000	1	8	Byte access
3'b001	2	16	Half-word
3'b010	4	32	Word
3'b011	8	64	Double-word
3'b100	16	128	Quad-word
3'b101	32	256	Octa-word
3'b110	64	512	512-bit
3'b111	128	1024	1024-bit

10.5 AXI3 vs AXI4 Comparison

Feature	AXI3	AXI4
Burst length	1-16	1-256 (INCR only for >16)
AWLEN/ARLEN width	4 bits	8 bits
Write data interleave	Yes (WID exists)	No (WID removed)
QoS signaling	No	Yes (AWQOS/ARQOS)
Region signaling	No	Yes (AWREGION/ARREGION)
User signals	No	Yes (AWUSER/WUSER/etc)
AWLOCK/ARLOCK width	2 bits	1 bit
Cache attributes	Basic	Enhanced
Write response rule	Can come early	Must wait for WLAST

---

11. Common Pitfalls & Best Practices

11.1 Pitfalls to Avoid

❌ VALID depending on READY

- Causes deadlock!
- VALID must be asserted independently

❌ Changing signals while VALID is HIGH

- Once VALID asserted, all associated signals MUST be stable

- Cannot retract or modify

❌ Assuming in-order completion

- Always check IDs!
- Different IDs can complete in any order

❌ Ignoring 4KB boundary

- Bursts crossing 4KB undefined behavior
- Master must split bursts

❌ Mismatched IDs

- BID must match AWID
- RID must match ARID
- Slave cannot change IDs

❌ Not handling backpressure

- Must support READY=0 from slave
- Cannot assume slave always ready

❌ Burst length off-by-one

- AWLEN=0 means 1 beat, not 0 beats!
- Number of transfers = AWLEN + 1

11.2 Best Practices

✅ Use wider data buses

- 128, 256, 512-bit buses reduce transactions
- Better utilization of DDR bandwidth

✅ Align bursts to DDR page size

- Maximize row buffer hits
- Reduce activate overhead

✅ Implement write buffering

- Collect small writes
- Issue efficient DDR bursts

✅ Use QoS for prioritization

- Real-time traffic gets high QoS

- Fair scheduling with age counters

✅ **Reorder aggressively**

- Bank parallelism
- Row buffer locality
- Minimize read/write turnaround

✅ **Add pipeline stages liberally**

- AXI designed for easy pipelining
- Register all signals for timing
- VALID/READY naturally supports this

✅ **Size bursts for DDR efficiency**

- Match DDR burst length (BL8)
- Align to cache line sizes
- Consider 4KB boundaries

✅ **Monitor performance**

- Track row buffer hit rate
- Measure average latency per QoS
- Identify bottlenecks

---

12. Transaction Examples

12.1 Simple Write Transaction

```
Cycle | AWVALID | AWREADY | AWADDR | AWLEN | WVALID | WREADY | WDATA |
WLAST | BVALID | BREADY |
------|---------|---------|--------|-------|--------|--------|-------|----
---|--------|--------|
  1   |    0    |    1    |   X    |   X   |   0    |   1    |   X   |   X
|   0    |   1    |
  2   |    1    |    1    | 0x1000 |   0   |   1    |   1    | 0xAA  |   1
|   0    |   1    | ← Address + Data
  3   |    0    |    1    |   X    |   X   |   0    |   1    |   X   |   X
|   1    |   1    | ← Response
  4   |    0    |    1    |   X    |   X   |   0    |   1    |   X   |   X
|   0    |   1    |
```

Analysis:

- Single-beat write (AWLEN=0)
- Address and data happen same cycle (both VALID/READY)
- Response comes next cycle
- Total: 2 cycles for complete transaction

12.2 Burst Write with Wait States

```
Cycle | AWVALID | AWREADY | AWADDR | AWLEN | WVALID | WREADY | WDATA |
WLAST | BVALID | BREADY |
------|---------|---------|--------|-------|--------|--------|-------|----
---|--------|--------|
  1   |    1    |    0    | 0x2000 |   2   |   0    |   1    |   X   |   0
|   0    |   1    | ← AW wait
  2   |    1    |    0    | 0x2000 |   2   |   1    |   1    | 0x11  |   0
|   0    |   1    | ← AW wait, W1
  3   |    1    |    1    | 0x2000 |   2   |   1    |   0    | 0x22  |   0
|   0    |   1    | ← AW done, W2 wait
  4   |    0    |    1    |   X    |   X   |   1    |   1    | 0x22  |   0
|   0    |   1    | ← W2 done
  5   |    0    |    1    |   X    |   X   |   1    |   1    | 0x33  |   1
|   0    |   1    | ← W3 done
  6   |    0    |    1    |   X    |   X   |   0    |   1    |   X   |   0
|   1    |   1    | ← Response
```

Analysis:

- 3-beat burst (AWLEN=2)
- Slave not ready for address (cycles 1-2)
- Slave not ready for 2nd data (cycle 3)
- Write data can proceed before address accepted
- Response after all data done

12.3 Out-of-Order Read Completion

```
Cycle | ARVALID | ARREADY | ARID | ARADDR | RVALID | RREADY | RID | RDATA
| RLAST |
------|---------|---------|------|--------|--------|--------|-----|-------
|-------|
  1   |    1    |    1    |  5   | 0x1000 |   0    |   1    |  X  |   X
|   X   | ← Read A
  2   |    1    |    1    |  7   | 0x2000 |   0    |   1    |  X  |   X
|   X   | ← Read B
  3   |    0    |    1    |  X   |   X    |   1    |   1    |  7  | 0xBB
|   1   | ← B returns first!
  4   |    0    |    1    |  X   |   X    |   1    |   1    |  5  | 0xAA
|   1   | ← A returns second
```

Analysis:

- Two reads issued back-to-back
- Read B (ARID=7) completes before Read A (ARID=5)
- Master uses RID to identify which data returned
- This is legal because IDs are different

12.4 Same ID Enforces Order

```
Cycle | ARVALID | ARREADY | ARID | ARADDR | RVALID | RREADY | RID | RDATA
| RLAST |
------|---------|---------|------|--------|--------|--------|-----|-------
|-------|
  1   |    1    |    1    |  3   | 0x1000 |   0    |   1    |  X  |   X
|   X   | ← Read A
  2   |    1    |    1    |  3   | 0x2000 |   0    |   1    |  X  |   X
|   X   | ← Read B (SAME ID!)
  3   |    0    |    1    |  X   |   X    |   0    |   1    |  X  |   X
|   X   | ← Wait...
  4   |    0    |    1    |  X   |   X    |   1    |   1    |  3  | 0xAA
|   1   | ← A MUST come first
  5   |    0    |    1    |  X   |   X    |   1    |   1    |  3  | 0xBB
|   1   | ← B comes second
```

Analysis:

- Both reads use ARID=3
- Even if B is faster, MUST wait for A
- Enforces in-order completion for same ID

---

13. Summary & Key Takeaways

For Memory Controller Design:

1. **Use AXI4** - Better burst support (256 beats), QoS, simpler writes
2. **Leverage out-of-order** - Issue multiple transactions, complete when ready
3. **Match DDR bursts** - Align AXI bursts to DDR BL8 for efficiency
4. **Reorder aggressively** - Bank parallelism, row buffer hits
5. **Use QoS** - Prioritize latency-sensitive traffic
6. **Buffer writes** - Combine small writes into efficient DDR bursts
7. **Handle backpressure** - Use ARREADY/AWREADY during refresh
8. **Pipeline everything** - AXI designed for registered stages
9. **Watch 4KB boundaries** - Aligns with DDR page size

10. **Monitor performance** - Track hit rates, latency, utilization

Critical Protocol Rules:

✅ VALID must NOT depend on READY
✅ Signals stable while VALID is HIGH
✅ Burst must NOT cross 4KB
✅ Number of beats = LEN + 1
✅ Response ID must match request ID
✅ Same ID = ordered, different ID = out-of-order
✅ WLAST/RLAST required on final beat
✅ All channels use VALID/READY handshake

---

14. AXI Interconnect Architecture (Your Block Diagram)

14.1 System Overview

Based on your memory controller block diagram:

```
[AXI M0] ──┐
           ├──> [Interconnect] ──> [MC] ──> [Standard UPHY IF] ──> [DDR
UPHY] ──> DDR0-5, LPDDR, etc.
[AXI M0] ──┘
```

Component roles:

- **AXI Masters (M0):** CPUs, DMA engines, peripherals issuing memory requests
- **Interconnect:** Routes transactions, arbitrates between masters, extends IDs
- **Memory Controller (MC):** Translates AXI to DDR protocol, handles reordering
- **UPHY (Universal PHY):** Physical interface to actual DRAM chips

14.2 AXI Interconnect Functions

The **green Interconnect block** performs critical functions:

14.2.1 Transaction Routing

Address Decoding:

- Each slave (MC in this case) has an address range
- Interconnect decodes AWADDR/ARADDR to determine target
- If address not mapped → generates DECERR response

Example:

```
MC address range: 0x8000_0000 - 0xFFFF_FFFF

Master transaction ARADDR=0x9000_0000 → Routes to MC
Master transaction ARADDR=0x1000_0000 → No slave, DECERR
```

14.2.2 Master Arbitration

When **multiple masters** request access simultaneously:

Common schemes:

1. **Fixed priority:** Master 0 always beats Master 1
2. **Round-robin:** Fair rotation
3. **QoS-based:** Use ARQOS/AWQOS from masters
4. **Age-based:** Oldest request wins (prevent starvation)

Your diagram has 2 masters → Interconnect must arbitrate

Example arbitration:

```
Cycle 1: M0 and M1 both assert ARVALID
         Interconnect grants M0 (higher priority)
         M0's ARVALID→MC, M1 waits

Cycle 2: M1's turn (round-robin)
         M1's ARVALID→MC
```

14.2.3 ID Width Extension

Problem: Multiple masters, each with their own transaction IDs

Solution: Interconnect **appends master ID bits**

Example:

```
Master 0: Uses ARID[3:0] (4 bits, 16 IDs)
Master 1: Uses ARID[3:0] (4 bits, 16 IDs)

Interconnect to MC: ARID[5:0] (6 bits)
  - ARID[5:4] = Master number (2 bits)
  - ARID[3:0] = Original master ID (4 bits)

Master 0, ID=5  → MC sees ID=0_0101 (decimal 5)
Master 1, ID=5  → MC sees ID=0_1101 (decimal 21)

When MC responds:
```

```
RID=0_0101 → Interconnect routes to Master 0, strips to RID=5
RID=0_1101 → Interconnect routes to Master 1, strips to RID=5
```

Why? Prevents ID collision between masters

14.2.4 Width Conversion (if needed)

If masters and MC have **different data widths:**

Upsizing (Master narrower than MC):

```
Master: 64-bit WDATA
MC: 128-bit WDATA

Interconnect combines 2 master beats into 1 MC beat:
  M beat 0: [63:0]   ─┐
  M beat 1: [63:0]   ─┼→ MC beat 0: [127:0]
```

Downsizing (Master wider than MC):

```
Master: 256-bit WDATA
MC: 128-bit WDATA

Interconnect splits 1 master beat into 2 MC beats:
  M beat 0: [255:0] → MC beat 0: [127:0], MC beat 1: [255:128]
```

14.2.5 Outstanding Transaction Management

Interconnect must track:

- Which master each outstanding transaction belongs to
- Transaction IDs for response routing
- Ordering requirements (same ID must stay ordered)

Typical implementation: Reorder buffer or transaction tracker

14.3 Memory Controller (MC) in Your System

The **MC block** (dark blue) receives AXI from interconnect and:

1. **Decodes address** → Bank, Row, Column for DDR
2. **Buffers transactions** → Write buffer, read queue
3. **Reorders for efficiency** → Row buffer hits, bank parallelism
4. **Generates DDR commands** → ACT, READ, WRITE, PRE, REF
5. **Manages timing** → tRCD, tRP, tRAS, tRFC, etc.
6. **Returns responses** → RDATA with RID, BRESP with BID

Interface to UPHY:

- Not AXI protocol!
- "Standard UPHY IF" = DDR command interface
- Commands: Activate, Read, Write, Precharge, Refresh
- Data: Controlled by UPHY timing

14.4 DDR UPHY (Universal PHY)

The **light blue DDR UPHY** block:

Function: Physical layer for DDR interface

- Converts parallel DDR commands → High-speed serial DDR protocol
- Handles DDR timing at Gbps rates (DDR4: 2133-3200 MT/s)
- Manages DQS strobes, termination, calibration
- Supports multiple DDR types (DDR3/4/5, LPDDR3/4/5)

DDR IF signals (right side):

- Multiple DDR channels (DDR0-5, LPDDR, LPDDR2, LPDDR3)
- Each connects to physical DRAM chips
- Rank/channel configuration

14.5 Data Flow Example

Complete flow for a read request:

```
1. AXI Master 0 issues read:
   ARADDR=0x9000_1000, ARLEN=7, ARSIZE=3, ARID=3

2. Interconnect:
   - Routes to MC (address matches)
   - Extends ID: ARID = 00_0011 (Master 0, ID 3)
   - Forwards to MC

3. Memory Controller:
   - Decodes: Bank 2, Row 0x900, Column 0x10
   - Checks: Bank 2 has different row open
   - Issues DDR commands via UPHY:
       PRE (precharge current row)
       ACT Bank 2, Row 0x900
       READ Bank 2, Column 0x10, Burst Length 8
   - Waits for data from DDR

4. DDR UPHY:
   - Sends commands to DDR3 chip on DDR IF
   - Captures read data from DRAM
```

```
   - Returns to MC

5. Memory Controller:
   - Receives 8 beats of data
   - Streams back on AXI:
       RDATA (8 beats), RID=00_0011, RRESP=OKAY, RLAST on beat 8

6. Interconnect:
   - Sees RID[5:4]=00 → Route to Master 0
   - Strips ID: RID=0011
   - Forwards to Master 0

7. AXI Master 0:
   - Receives data, matches ARID=3
   - Transaction complete!
```

14.6 Key Considerations for Your Design

Interconnect:

- Choose arbitration scheme (QoS-aware recommended)
- Size ID width extension (2 bits for 4 masters, 3 bits for 8, etc.)
- Pipeline stages for timing
- Outstanding transaction limit per master

Memory Controller:

- Write buffer depth (32-64 entries typical)
- Read reorder buffer (16-32 entries)
- Page policy: Open-page vs Close-page
- Refresh scheduling (all-bank vs per-bank)
- QoS scheduler for ARQOS/AWQOS

UPHY Interface:

- Standard interface depends on PHY vendor
- Usually DDR command + data buses
- Timing constraints critical (setup/hold)

Performance tuning:

- Monitor row buffer hit rate (target >70%)
- Track average latency per QoS level
- Measure DDR bandwidth utilization
- Identify bottlenecks (interconnect vs MC vs DDR)

14.7 Why This Architecture?

Advantages:

1. **Scalability:** Easy to add more masters
2. **Flexibility:** Interconnect isolates masters from MC complexity
3. **Reusability:** MC and UPHY can be standard IP blocks
4. **Performance:** Out-of-order completion hides DDR latency
5. **Multi-channel:** UPHY can drive multiple DDR channels for bandwidth

Your system supports:

- Multiple DDR types (DDR3/4/5, LPDDR variants)
- Multiple masters (CPUs, DMA, GPUs, etc.)
- High bandwidth (parallel channels)
- QoS differentiation (real-time vs best-effort)
